



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

CAMPUS D'ALCOI



DEPARTAMENTO DE SISTEMAS
INFORMÁTICOS Y COMPUTACIÓN

Cloud computing

Laboratorio 2

Contenido

1. Introducción.....	3
2. Gestión de imágenes.....	3
3. Gestión de usuarios.....	4
4. Control de accesos.....	5
5. Compartición de recursos.....	7
6. Gestión de cuotas de usuarios.....	8
7. Servicio RESTful.....	9
8. El CLI.....	11

1. Introducción

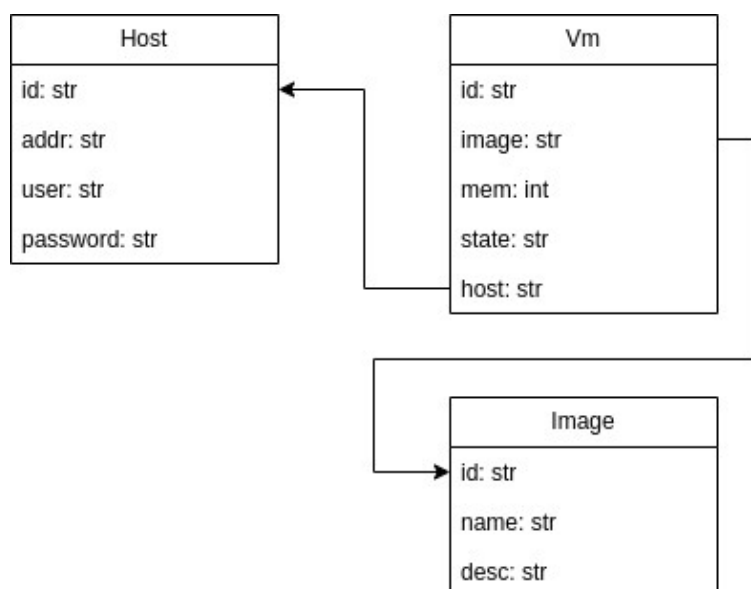
En este laboratorio continuamos mejorando el IaaS que se comenzó en el laboratorio 1. En concreto, se plantea incorporar las siguientes funcionalidades:

- Gestión de imágenes
- Control de usuarios
- Control de accesos
- Compartición de recursos
- Gestión de cuotas de usuario
- Publicación de funcionalidades por medio de un servicio RESTful
- CLI

En las siguientes secciones elaboramos estas funcionalidades.

2. Gestión de imágenes

En el laboratorio anterior se asumió la existencia de imágenes en el repositorio del IaaS. Llegó el momento de mejorar este aspecto. Para ello, se propone ampliar el modelo de datos del IaaS tal y como se describe en el siguiente diagrama.



Como se puede observar, la base de datos recogerá toda la información sobre las imágenes soportadas en el sistema. Será por tanto necesario ampliar la estructura de la base de datos, añadiendo la creación de una nueva tabla a la función *init()* definida en el laboratorio anterior.

Además, será necesario añadir nuevas operaciones de gestión de imágenes al módulo *core.py*. Se proponen las siguientes:

Operación	Descripción
<code>addImage(url:str, img:dict)</code>	Añade una nueva imagen al IaaS. El parámetro <i>url</i> especifica su ubicación origen. El parámetro <i>img</i> es un diccionario que contiene al menos los campos <i>name</i> y <i>desc</i> .
<code>saveVmAsImage(vmId:str, img:dict)</code>	Crea una nueva imagen a partir de una vm existente. El parámetro <i>vmId</i> especifica la vm. El parámetro <i>img</i> es un diccionario que contiene al menos los campos <i>name</i> y <i>desc</i> .

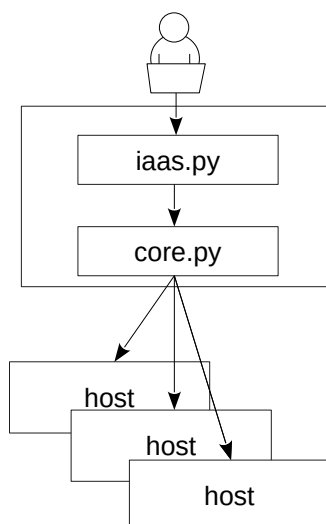
	Si la vm está en ejecución, se recomienda pararla previamente, para evitar pérdidas de datos en la imagen.
<code>removeImage(imgId)</code>	Elimina la imagen especificada.
<code>listImages(query:str='')</code>	Lista las imágenes que verifican el filtro especificado en <i>query</i> .

Es necesario recordar que las imágenes se almacenan en el repositorio `/export/iaas/images` exportado en el IaaS. Se propone guardar las imágenes como ficheros `<imgId>.qcow2`.

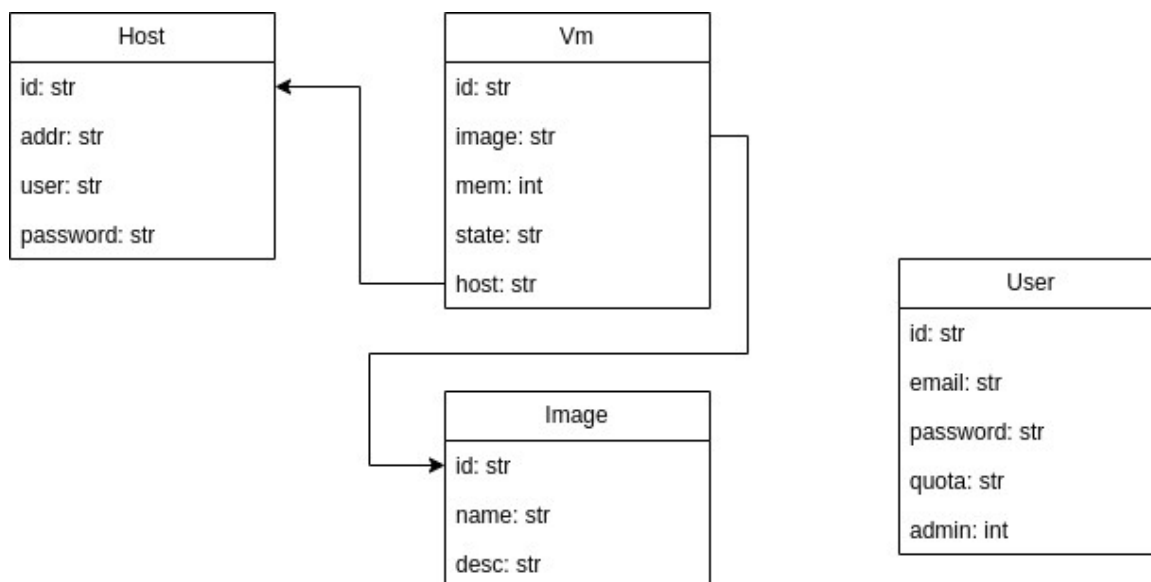
Para obtener una imagen a partir de una url, se recomienda utilizar la función `urllib.request.urlretrieve()`.

3. Gestión de usuarios

Hasta este punto, se dispone de un IaaS que permite gestionar recursos (hosts, vms e imágenes) a un nivel muy básico, sin ningún tipo de gestión de usuarios, ni control de accesos. Se propone recubrirlo con un nuevo módulo, que denominaremos *iaas.py*, que utilizará las funcionalidades del módulo *core.py*, y que añadirá funcionalidades avanzadas como gestión de usuarios, control de accesos, compartición de recursos, etc. La arquitectura del IaaS resultante sería la siguiente:



Para implementar la gestión de usuarios, será necesario ampliar el modelo de datos que maneja el IaaS, tal y como se refleja en el siguiente diagrama.



Como se puede observar, un usuario puede ser administrador. Un usuario administrador podrá acceder a todas las funcionalidades y a todos los recursos del IaaS. En concreto, sólo un administrador podrá:

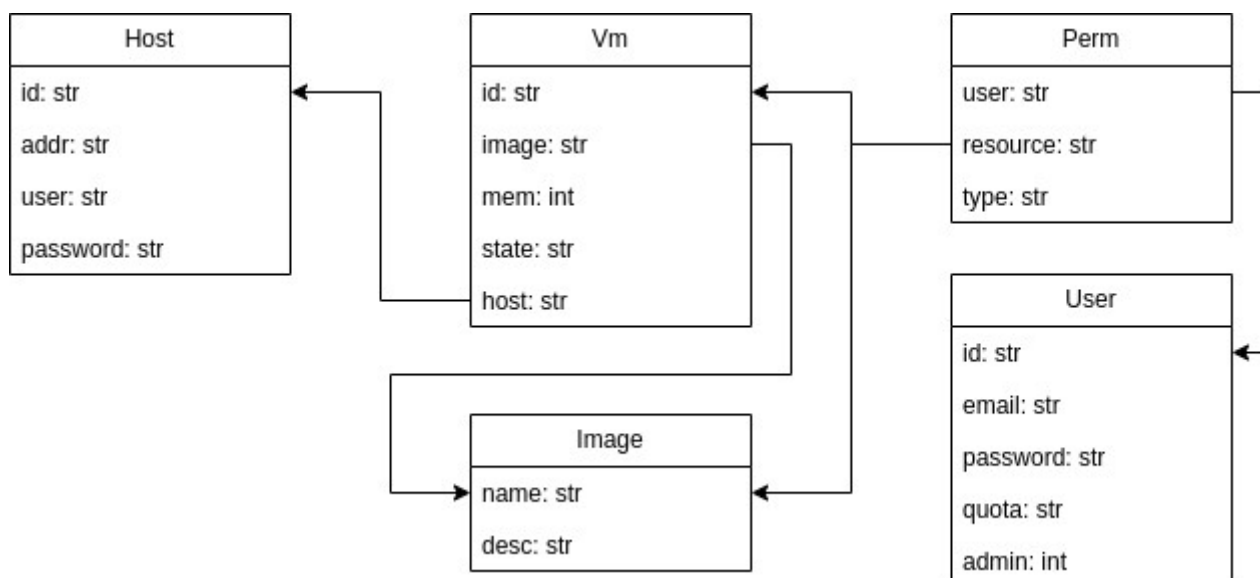
- Añadir y eliminar usuarios.
- Añadir y eliminar hosts.

Se propone implementar las siguientes operaciones en el nuevo módulo *iaas.py*.

<u>Operación</u>	<u>Descripción</u>
init()	Inicializa el core del IaaS, invocando la función <i>core.init()</i> y crea las tablas necesarias para añadir funcionalidades avanzadas. Entre otras, la tabla <i>users</i> .
login(email:str, password:str)	Autentica a un usuario. Si la operación tiene éxito, devuelve un token cifrado, que contiene al menos el id del usuario autenticado y la fecha de autenticación. El resto de operaciones de <i>iaas.py</i> debe aceptar como primer parámetro este token.
validateToken(token:str)	Valida el token especificado. Si la operación tiene éxito devuelve la información del usuario autenticado.
addUser(token:str, user:dict)	Crea un nuevo usuario. El parámetro <i>user</i> es un diccionario que al menos contiene <i>email</i> y <i>password</i> . Esta operación sólo puede ser invocada por un administrador. Si la operación tiene éxito, devuelve el usuario creado.
removeUser(token:str, userId:str)	Elimina el usuario especificado. Sólo puede ser invocada por un administrador. Podría conllevar la destrucción de todos los recursos asignados al usuario.
updateUser(token:str, userId:str, data:dict)	Actualiza el usuario especificado. Sólo puede ser invocada por un administrador o el propio usuario.
listUsers(token:str, query:str='')	Lista los usuarios especificados en el filtro <i>query</i> .

4. Control de accesos

En un IaaS serio los recursos están compartimentados en cuentas, y cada cuenta únicamente puede acceder a sus recursos. Vamos a implementar esta funcionalidad implantando un sistema de permisos en nuestro modelo de datos.



Como se puede observar, la entidad *Perm* representa un permiso que posee un usuario sobre un recurso. En principio, los recursos que maneja nuestro IaaS son imágenes y vms. Los hosts sólo pueden ser gestionados por un administrador. Con este mecanismo ya resulta sencillo conocer quién tiene acceso a qué. Siempre que un usuario no administrador acceda a imágenes o vms, primero será necesario consultar sus permisos. Un administrador siempre puede acceder a cualquier recurso.

Será necesario implementar las siguientes operaciones en el módulo *iaas.py*.

Operación	Descripción
<code>addHost(token:str, host:dict)</code>	Un administrador añade un host. Si es administrador, se invocará a <code>core.addHost()</code> .
<code>listHosts(token:str, query:str='')</code>	Cualquier usuario puede listar los hosts. Se invocará <code>core.listHosts()</code> .
<code>updateHost(token:str, hostId:str, data:dict)</code>	Un administrador actualiza un host. Si es administrador, se invocará <code>core.updateHost()</code> .
<code>removeHost(token:str, hostId:str)</code>	Un administrador elimina un host. Si es administrador, se invocará a <code>core.removeHost()</code> .
<code>addVm(token:str, vm:dict)</code>	Un usuario añade una vm a partir de una imagen. Si el usuario no es administrador debe tener permisos sobre la imagen origen. Si es el caso, entonces se invoca <code>core.addVm()</code> . Al finalizar será necesario añadir un nuevo permiso del usuario creador sobre la vm creada.
<code>listVms(token:str, query:str='')</code>	Un usuario lista sus vms. Si el usuario no es administrador, será necesario obtener sus permisos sobre vms. Después invocará <code>core.listVms()</code> buscando dichas vms.
<code>startVm(token:str, vmId:str)</code>	Un usuario arranca su vm. Si el usuario no es administrador, será necesario comprobar su permiso sobre la vm. Después invocará <code>core.startVm()</code> .
<code>stopVm(token:str, vmId:str)</code>	Un usuario para su vm. Si el usuario no es administrador, será necesario comprobar su permiso sobre la vm. Después invocará <code>core.stopVm()</code> .
<code>removeVm(token:str, vmId:str)</code>	Un usuario elimina su vm. Si el usuario no es administrador, será necesario comprobar su permiso sobre la vm. Después

	invocará <i>core.removeVm()</i> . Deberán eliminarse todos los permisos existentes sobre la vm.
<i>saveVmAsImage(token:str, vmId:str, img:dict)</i>	Un usuario guarda su vm como nueva imagen. Si el usuario no es administrador, será necesario comprobar su permiso sobre la vm. Después invocará <i>core.saveVmAsImage()</i> . Deberá añadirse un nuevo permiso del usuario sobre la imagen creada.
<i>addImage(token:str, url:str, img:dict)</i>	Un usuario añade una imagen. Para ello, invoca <i>core.addImage()</i> . Al finalizar será necesario añadir un nuevo permiso del usuario creador sobre la imagen creada.
<i>listImages(token:str, query:str='')</i>	Un usuario lista sus imágenes. Si el usuario no es administrador, será necesario obtener sus permisos sobre imágenes. Después invocará <i>core.listImages()</i> buscando dichas imágenes.
<i>removeImage(token:str, imgId:str)</i>	Un usuario elimina su imagen. Si el usuario no es administrador, será necesario comprobar su permiso sobre la imagen. Después invocará <i>core.removeImage()</i> . Deberán eliminarse todos los permisos existentes sobre la imagen.

5. Compartición de recursos

Una vez implantado el sistema de permisos descrito en el apartado anterior, resulta sumamente sencillo compartir recursos entre distintos usuarios. En este escenario, compartir un recurso únicamente implicará crear un permiso adicional. De este modo, se propone implementar las siguientes operaciones en el módulo *iaas.py*:

<u>Operación</u>	<u>Descripción</u>
<i>shareVm(token:str, vmId:str, userId:str)</i>	Un usuario comparte su vm. Si el usuario no es administrador, será necesario comprobar su permiso sobre la vm. Después creará el permiso especificado.
<i>unshareVm(token:str, vmId:str, userId:str)</i>	Un usuario deja de compartir su vm. Si el usuario no es administrador, será necesario comprobar su permiso sobre la vm. Después eliminará el permiso especificado.
<i>listVmShares(token:str, vmId:str)</i>	Lista los permisos sobre una vm. Si el usuario no es administrador, será necesario comprobar su permiso sobre la vm.
<i>shareImage(token:str, imgId:str, userId:str)</i>	Un usuario comparte su imagen. Si el usuario no es administrador, será necesario comprobar su permiso sobre la imagen. Después creará el permiso especificado.
<i>unshareImage(token:str, imgId:str, userId:str)</i>	Un usuario deja de compartir su imagen. Si el usuario no es administrador, será necesario comprobar su permiso sobre la imagen. Después eliminará el permiso especificado.
<i>listImageShares(token:str, imgId:str)</i>	Lista los permisos sobre una imagen. Si el usuario no es administrador, será necesario comprobar su permiso sobre la imagen.

6. Gestión de cuotas de usuarios

En un IaaS serio cada cuenta posee unos límites de recursos que no puede superar. En esta sección se propone diseñar un sistema de cuotas que permita especificar estos límites, sobre cada usuario, y que el IaaS compruebe que los límites no se exceden. En caso de ser excedidos, el IaaS rechazará las operaciones que involucren reserva de nuevos recursos.

A continuación se listan algunas métricas que se podrían tener en cuenta en la cuotas:

- Número de imágenes creadas.
- Número de vms creadas.
- Tiempo de CPU consumido.
- Memoria consumida.
- Bytes leídos, bytes escritos en disco.
- Bytes leídos, bytes escritos en red.
- etc.

Es sencillo recoger muchas de estas métricas desde Libvirt, utilizando para ello un código parecido al siguiente, que ha sido extraído de la siguiente URL:

<https://libvirt-python.readthedocs.io/monitoring-performance/>

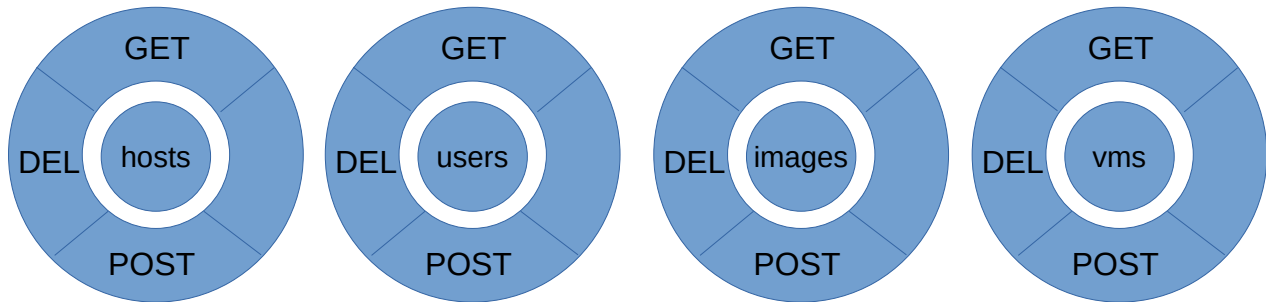
```
import libvirt

conn = libvirt.open("qemu:///system")
dom = conn.lookupByID(1)
cpu_stats = dom.getCPUStats(False)
for (i, cpu) in enumerate(cpu_stats):
    print("CPU {} Time: {}".format(i, cpu["cpu_time"] / 1000000000.))
stats = dom.getCPUStats(True)
print("cpu_time:      {cpu_time}\n"
      "system_time: {system_time}\n"
      "user_time:    {user_time}".format(**stats[0]))
stats = dom.memoryStats()
print("memory used:")
for name in stats:
    print("{} ({}).format(stats[name], name))

tree = ElementTree.fromstring(dom.XMLDesc())
iface = tree.find("devices/interface/target").get("dev")
stats = dom.interfaceStats(iface)
print("read bytes:      {}\n"
      "read packets:   {}\n"
      "read errors:     {}\n"
      "read drops:      {}\n"
      "write bytes:     {}\n"
      "write packets:   {}\n"
      "write errors:    {}\n"
      "write drops:     {}".format(*stats))
```

7. Servicio RESTful

En esta sección se propone publicar toda la funcionalidad del IaaS por medio de un servicio RESTful. En concreto, el servicio proporcionará acceso a 4 colecciones de recursos principales: hosts, usuarios, imágenes y vms.

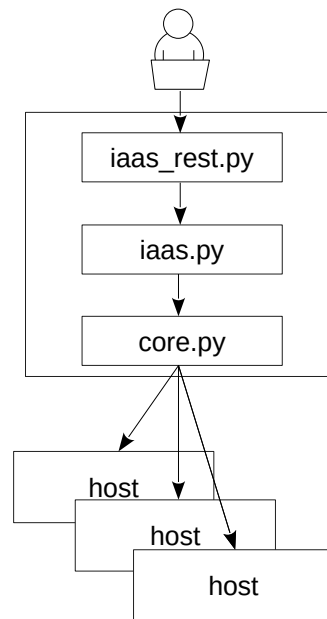


A continuación se resume la API del servicio RESTful a implementar:

Método	URL	Comentarios
Recurso sessions		
POST	/iaas/sessions	Crea una sesión. <u>Contenido (JSON)</u> : {email,password} <u>Resultado (text)</u> : token
Recurso hosts		
GET	/iaas/hosts	Lista los hosts. <u>Cabeceras</u> : Authorization:token <u>Parámetros</u> : query <u>Resultado (JSON)</u> : [host]
POST	/iaas/hosts	Crea un host. <u>Cabeceras</u> : Authorization:token <u>Contenido (JSON)</u> : host <u>Resultado (JSON)</u> : host
PUT	/iaas/hosts/<hostId>	Actualiza un host. <u>Cabeceras</u> : Authorization:token <u>Contenido (JSON)</u> : data
DELETE	/iaas/hosts/<hostId>	Elimina un host <u>Cabeceras</u> : Authorization:token
Recurso users		
GET	/iaas/users	Lista los usuarios. <u>Cabeceras</u> : Authorization:token <u>Parámetros</u> : query <u>Resultado (JSON)</u> : [user]
POST	/iaas/users	Crea un usuario. <u>Cabeceras</u> : Authorization:token <u>Contenido (JSON)</u> : user <u>Resultado (JSON)</u> : user
PUT	/iaas/users/<userId>	Actualiza un usuario. <u>Cabeceras</u> : Authorization:token <u>Contenido (JSON)</u> : data
DELETE	/iaas/users/<userId>	Elimina un usuario.

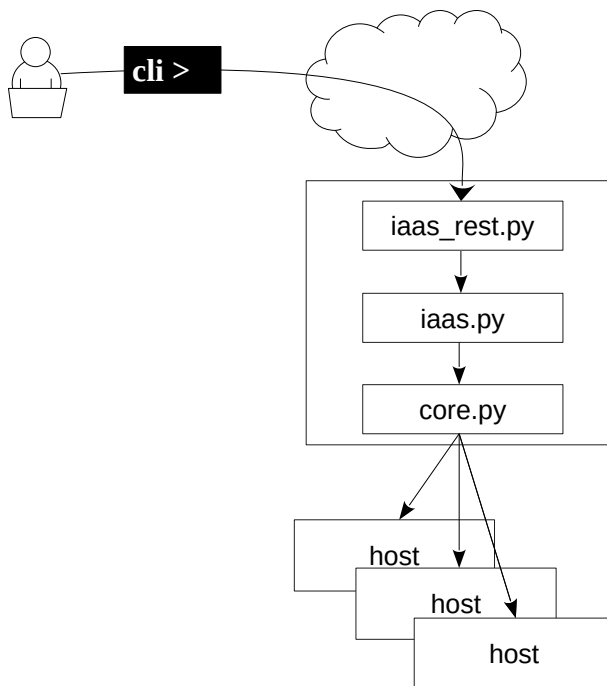
		<u>Cabeceras</u> : Authorization:token
Recurso imágenes		
GET	/iaas/images	Lista las imágenes. <u>Cabeceras</u> : Authorization:token <u>Parámetros</u> : query <u>Resultado</u> (JSON): [image]
POST	/iaas/images	Crea una imagen. <u>Cabeceras</u> : Authorization:token <u>Contenido</u> (JSON): image <u>Resultado</u> (JSON): image
PUT	/iaas/images/<imgId>	Actualiza una imagen. <u>Cabeceras</u> : Authorization:token <u>Contenido</u> (JSON): data
DELETE	/iaas/images/<imgId>	Elimina una imagen. <u>Cabeceras</u> : Authorization:token
GET	/iaas/images/<imgId>/shares	Lista los permisos sobre una imagen. <u>Cabeceras</u> : Authorization:token <u>Resultado</u> (JSON): [perm]
POST	/iaas/images/<imgId>/shares	Crea un permiso sobre una imagen. <u>Cabeceras</u> : Authorization:token <u>Contenido</u> (JSON): {user}
DELETE	/iaas/images/<imgId>/shares/<userId>	Elimina un permiso sobre una imagen. <u>Cabeceras</u> : Authorization:token
Recurso vms		
GET	/iaas/vms	Lista las vms. <u>Cabeceras</u> : Authorization:token <u>Parámetros</u> : query <u>Resultado</u> (JSON): [vm]
POST	/iaas/vms	Crea una vm. <u>Cabeceras</u> : Authorization:token <u>Contenido</u> (JSON): vm <u>Resultado</u> (JSON): vm
PUT	/iaas/vms/<vmId>	Actualiza una vm. <u>Cabeceras</u> : Authorization:token <u>Contenido</u> (JSON): data
DELETE	/iaas/vms/<vmId>	Elimina una vm. <u>Cabeceras</u> : Authorization:token
GET	/iaas/vms/<vmId>/shares	Lista los permisos sobre una vm. <u>Cabeceras</u> : Authorization:token <u>Resultado</u> (JSON): [perm]
POST	/iaas/vms/<vmId>/shares	Crea un permiso sobre una vm. <u>Cabeceras</u> : Authorization:token <u>Contenido</u> (JSON): {user}
DELETE	/iaas/vms/<vmId>/shares/<userId>	Elimina un permiso sobre una vm. <u>Cabeceras</u> : Authorization:token

La API será implementada en el módulo *iaas_rest.py*, utilizando *Flask*, dando lugar a la siguiente arquitectura.



8. El CLI

Por último, el servicio RESTful será accedido por un CLI, que suministrará una interfaz basada en consola, y que hará uso de la librería *requests*.



A continuación se muestra un posible listado de opciones disponibles a través de esta herramienta:

```
Usage: python cli.py
Welcome!
cli> help
Available commands:
  help
  exit
  login <email> <password>
  host ls [<query>]
  host add <addr> <user> <password>
  host rm <hostId>
  user list [<query>]
  user add <email> <password>
  user rm <userId>
  image ls [<query>]
  image add <url> <name> <desc>
  image rm <imgId>
  image shares <imgId>
  image share <imgId> <userId>
  image unshare <imgId> <userId>
  vm ls [<query>]
  vm add <image> [<mem>]
  vm start <vmId>
  vm stop <vmId>
  vm rm <vmId>
  vm save <vmId> <imgName> <imgDesc>
  vm shares
  vm share <vmId> <userId>
  vm unshare <vmId> <userId>
```